

Лабораторная работа № 1

Начало работы с MySQL. MySQL Workbench

Задание 1

Раздел «Management»:

1. Раздел Server status (описано в примере)
2. Раздел Client Connections. Назначение: Отслеживание и управление текущими подключениями к серверу баз данных.
 - a. Просмотр списка всех активных подключений.
 - b. Информация о соединении: ID процесса, пользователь, хост, с которого выполнено подключение, база данных, время работы запроса, текущий выполняемый SQL-запрос.
 - c. Возможность принудительно завершить соединения, которые потребляют много ресурсов или блокируют другие операции
3. Раздел Users and Privileges. Централизованное управление учетными записями пользователей и их правами доступа к базам данных.
 - a. Список пользователей: Просмотр всех зарегистрированных пользователей и хостов, с которых они могут подключаться.
 - b. Добавление и удаление учетных записей.
 - c. Управление правами доступа:
 - d. Назначение глобальных ролей (например только чтение или только запись).
 - e. Ограничение на количество запросов или соединений в час для пользователя.
 - f. Детальная настройка прав для каждого пользователя на каждую базу данных и таблицу.
4. Раздел Status and System Variables. Просмотр и изменение конфигурации сервера MySQL.
 - a. Статистические данные, которые показывают текущее состояние сервера
 - b. Настройки конфигурации сервера
5. Раздел Data Export. Экспорт данных и структуры баз данных в файлы.
 - a. Выбор конкретных баз данных и таблиц для экспорта.
 - b. Выбор формата экспорта: SQL-файл, CSV, JSON, XML и др.
 - c. Возможность создать один файл для каждой таблицы или один общий файл.
 - d. Создание единого SQL-файла, содержащего всё необходимое для восстановления базы на другом сервере.
6. Раздел Data Import/Restore. Импорт данных из ранее созданных SQL-файлов для восстановления или переноса базы данных.
 - a. Указание пути к файлу

- b. Выбор схемы (базы данных), в которую будут импортироваться данные.
- c. Возможность настроить режим обработки ошибок (пропускать или останавливаться).

Раздел «Instance»

1. Раздел Startup\Shutdown. Управление жизненным циклом сервера MySQL.
 - a. Startup: Запуск сервера MySQL (если он остановлен). Требуется прав администратора на операционной системе.
 - b. Shutdown: Корректная остановка сервера. Сервер завершает текущие транзакции и корректно закрывает все соединения перед остановкой. Это предпочтительный метод против принудительного убийства процесса.
2. Раздел Server Logs. Просмотр журналов (логов) сервера MySQL в удобном интерфейсе.
 - a. Error Log: Содержит информацию об ошибках запуска, критических сбоях и предупреждениях.
 - b. General Query Log: Записывает все запросы, приходящие на сервер.
 - c. Slow Query Log: Записывает запросы, выполнение которых превысило заданное время.
 - d. Binary Log: Записывает все изменения данных.
3. Раздел Options File. Визуальный редактор основного конфигурационного файла MySQL.
 - a. Просмотр и изменение параметров конфигурации сервера, сгруппированных по разделам.

Раздел «Performance»

1. Раздел Dashboard. Визуальная панель управления с ключевыми метриками производительности в реальном времени.
 - a. MySQL Connections: График количества соединений.
 - b. Query Throughput: Пропускная способность запросов (запросов в секунду).
 - c. Query Execution Time: Время выполнения запросов.
 - d. Network Traffic In/Out: Сетевой трафик.
 - e. Statement Latency: Задержки выполнения различных типов SQL-запросов (SELECT, INSERT, UPDATE, DELETE).
2. Раздел Performance Reports. Генерация готовых отчетов о производительности на основе данных из Performance Schema (если он включен).
 - a. Автоматический анализ и вывод рекомендаций.
 - b. Помогает быстро идентифицировать самые медленные запросы или операции, потребляющие больше всего времени CPU/I/O.
3. Раздел Performance Schema Setup. Настройка подсистемы Performance Schema — механизма для сбора статистики о работе сервера.
 - a. Включение или отключение Performance Schema.
 - b. Управление инструментами: можно выбрать, какие именно метрики собирать (например, только ошибки и т.д.), чтобы минимизировать нагрузку на производительность от самого мониторинга.
 - c. Просмотр текущего состояния и конфигурации инструментов.

Задания 2, 3

Запрос, соответствующий созданию таблицы simpledb:

```
CREATE DATABASE simpledb /*!40100 DEFAULT CHARACTER SET utf8mb3 */  
/*!80016 DEFAULT ENCRYPTION='N' */;
```

Задание 4

```
INSERT INTO simpledb.users (id, name, email) VALUES ('', 'Alexey', 'Alex@mail.ru');  
INSERT INTO simpledb.users (name, email) VALUES ('Michael', 'm@m.com');  
INSERT INTO simpledb.users (name, email) VALUES ('Nikolay', 'Nik@inbox.ru');
```

Задание 5

```
ALTER TABLE simpledb.users  
ADD COLUMN gender ENUM('M', 'F') NULL AFTER email,  
ADD COLUMN bday DATE NULL AFTER gender,  
ADD COLUMN postal_code VARCHAR(10) NULL AFTER bday,  
ADD COLUMN rating FLOAT NOT NULL AFTER postal_code,  
ADD COLUMN created TIMESTAMP NULL DEFAULT CURRENT_TIMESTAMP() AFTER  
rating,  
CHANGE COLUMN name name VARCHAR(50) NOT NULL ;
```

Параметр по умолчанию «CURRENT_TIMESTAMP()» указывает текущую дату и текущее время, если в поле TIMESTAMP не было введено иных данных. При выполнении работы, параметр «NOT NULL» использовался для полей id, name, email и rating.

Задания 6, 7

```
/*  
-- Query: SELECT * FROM simpledb.users  
LIMIT 0, 1000  
  
-- Date: 2025-09-15 19:03  
*/  
INSERT INTO `` (id,name,email,gender,bday,postal_code,rating,created) VALUES  
(1,'Alexey','Alex@mail.ru',NULL,NULL,NULL,0,'2025-09-15 18:53:25');  
INSERT INTO `` (id,name,email,gender,bday,postal_code,rating,created) VALUES  
(2,'Michael','m@m.com',NULL,NULL,NULL,0,'2025-09-15 18:53:25');
```

```
INSERT INTO `` (id,name,email,gender,bday,postal_code,rating,created) VALUES
(3,'Nikolay','Nik@inbox.ru',NULL,NULL,NULL,0,'2025-09-15 18:53:25');
INSERT INTO `` (id,name,email,gender,bday,postal_code,rating,created) VALUES
(5,'Alexa','amazon@gmail.com','F','2014-06-11','111111',12,'2025-09-15
18:55:41');
INSERT INTO `` (id,name,email,gender,bday,postal_code,rating,created) VALUES
(6,'Oleg','tbank@yandex.ru','M','2021-11-03','123876',9,'2025-09-15 18:55:41');
INSERT INTO `` (id,name,email,gender,bday,postal_code,rating,created) VALUES
(7,'Ekaterina','ekaterina.petrova@outlook.com','F','2000-02-
11','145789',1.123,'2025-09-15 18:59:55');
INSERT INTO `` (id,name,email,gender,bday,postal_code,rating,created) VALUES
(8,'Paul','paul@superpochta.ru','M','1998-08-12','123789',1,'2025-09-15
18:59:55');
```

Задание 8

```
CREATE TABLE simplifiedb.resume (
  resumeid INT NOT NULL AUTO_INCREMENT,
  userid INT NOT NULL,
  title VARCHAR(100) NOT NULL,
  skills TEXT NULL,
  created TIMESTAMP NULL DEFAULT CURRENT_TIMESTAMP(),
  PRIMARY KEY (resumeid),
  INDEX userid_idx (userid ASC) VISIBLE,
  CONSTRAINT userid
  FOREIGN KEY (userid)
  REFERENCES simplifiedb.users (id)
  ON DELETE CASCADE
  ON UPDATE CASCADE);
```

Столбцы userid в таблице resume и id в таблице users теперь связаны. Если изменить параметр id в таблице users, параметр userid с таким же значением в таблице resume также изменится. Аналогично, удаление параметра в одной таблице приведет в автоматическому его удалению в другой. Однако, это работает только в одну сторону (при изменении\удалении значения из столбца userid, значение в столбце id таблицы users не изменится).

Задание 9

Так как для userid не задано значение «unique», у одного пользователя может быть сколько угодно резюме (от нуля)

```
/*  
-- Query: SELECT * FROM simplifiedb.resume  
LIMIT 0, 1000  
  
-- Date: 2025-09-15 19:34  
*/  
INSERT INTO `` (resumeid,userid,title,skills,created) VALUES (1,2,'Я хороший','soft  
skills','2025-09-15 19:26:21');  
INSERT INTO `` (resumeid,userid,title,skills,created) VALUES (2,3,'Топ 100 причин,  
почему меня нужно взять на работу','HTML, CSS, JS','2025-09-15 19:26:21');  
INSERT INTO `` (resumeid,userid,title,skills,created) VALUES (3,1,'Сочинение на  
тему...','Not Found','2025-09-15 19:26:21');  
INSERT INTO `` (resumeid,userid,title,skills,created) VALUES (4,2,'Еще одно  
резюме','Gaming','2025-09-15 19:28:33');
```

Попытка добавить резюме с id несуществующего пользователя вызывает ошибку «Cannot add or update a child row: foreign key constraint fails»

Задание 10

После удаления из таблицы users пользователя с id=2, из таблицы resume автоматически удалились все строки с userid=2

```
DELETE FROM simplifiedb.users WHERE (id = '2');
```

Result Grid

Filter Rows:

	id	name
	1	Alexey
▶	3	Nikolay
	5	Alexa
	6	Oleg
	7	Ekaterina
	8	Paul

	resumeid	userid	title	skills	created
	2	3	Топ...	HT...	2025-0...
▶	3	1	Соч...	Not ...	2025-0...
✱	NULL	NULL	NULL	NULL	NULL

Выше представлено состояние таблиц до изменения. При изменении id одного из пользователей в таблице users, в таблице resume меняется аналогичный userid.

```
UPDATE simplifiedb.users SET id = '256' WHERE (id = '3');
```

	id	name
	1	Alexey
▶	5	Alexa
	6	Oleg
	7	Ekaterina
	8	Paul
	256	Nikolay
★	NULL	NULL

	resumeid	userid	title	skills	created
	2	256	Ton...	HT...	2025-0...
▶	3	1	Соч...	Not ...	2025-0...
★	NULL	NULL	NULL	NULL	NULL